

Relatório Técnico – Evolução do Código do Elevador

1. Introdução

O projeto consiste no desenvolvimento de um sistema de controlo de um elevador didático utilizando um microcontrolador Arduino.

O objetivo é simular o funcionamento de um elevador de quatro pisos, incluindo:

- pedidos de andares através de botões,
- indicação luminosa dos pedidos,
- indicação da posição do elevador,
- simulação do movimento do motor através de uma ponte H,
- feedback sonoro com buzzer,
- apresentação de informação num display LCD.

Durante os testes em simulação (Tinkercad) foram identificados vários comportamentos incorretos relacionados com:

- gestão de múltiplos pedidos de andares,
- LEDs de indicação de chamada,
- e toque do buzzer em cada paragem.

Este relatório descreve:

- o erro inicial,
- as alterações implementadas,
- e a justificação das soluções adotadas (incluindo o uso de arrays como sugerido pelo professor).

2. Problema Inicial: Gestão Incorreta de Múltiplos Pedidos

2.1 Funcionamento original

No código original, os pedidos de andares eram tratados diretamente desta forma:

```
//Controle dos botões
if (!digitalRead(pinBotaoA)) andarDestino = 1;
if (!digitalRead(pinBotaoB)) andarDestino = 2;
if (!digitalRead(pinBotaoC)) andarDestino = 3;
if (!digitalRead(pinBotaoD)) andarDestino = 4;
```

Ou seja:

- Cada vez que um botão era pressionado, a variável `andarDestino` era atualizada com o valor desse andar.
- **Não existia nenhuma estrutura para guardar vários pedidos em simultâneo.**
- `andarDestino` guardava **apenas um único pedido: o último.**

2.2 Explicação do erro, com exemplo concreto

O problema torna-se evidente quando existem **múltiplas chamadas**.
Por exemplo:

1. O elevador está parado no **piso 1**.
2. O utilizador carrega no botão do **piso 3**.
 - `andarDestino` passa a ser 3.
 - O LED do piso 3 acende (chamada registada).
3. Em seguida, o utilizador carrega no botão do **piso 4**.
 - `andarDestino` passa a ser 4.
 - O pedido do piso 3 deixa de estar representado na variável de destino, porque foi substituído pelo novo valor.
4. O elevador começa a subir:
 - Passa pelo **piso 2** (se existisse sensor),
 - Chega ao **piso 3**, mas como `andarDestino` já está em 4, o sistema **não trata o piso 3 como destino final**, apenas como “ponto de passagem”.
5. Quando o elevador chega ao **piso 4**:
 - O código faz parar o movimento,
 - apaga os LEDs de chamada,
 - e considera o pedido atendido.

Na prática, isto gera dois problemas:

- O pedido do piso 3 não é atendido como deveria → o elevador não pára logicamente “para satisfazer” o pedido inicial.
- Os LEDs não refletem corretamente os pedidos:
 - o LED do 3 pode permanecer ligado até chegar ao 4,
 - e depois, ao atender o 4, o sistema acaba por apagar **vários LEDs ao mesmo tempo**, dando a sensação de que só o último pedido foi realmente considerado.

Em resumo:

O código original só guardava um único pedido (o último), através da variável `andarDestino`.

Assim, sempre que um novo botão era pressionado, o pedido anterior era perdido.

Este comportamento não corresponde ao de um elevador real, onde:

- cada pedido é registado,
- o elevador pára sucessivamente nos pisos pedidos,
- e à medida que vai passando por cada andar pedido, esse pedido é removido e a indicação luminosa é apagada.

3. Solução Implementada: Utilização de um Array de Pedidos

3.1 Variável intermédia para guardar os andares pedidos

Com base na sugestão do professor:

“para poderes guardar mais que um andar, tens de criar uma variável intermédia (...) Eu aconselhei-te um array pois penso que seria o mais fácil e explícito”

foi criada uma estrutura intermédia para armazenar os pedidos de andares:

```
// Pedidos de andares: pedidos[i] = true significa que o andar (i+1)
foi pedido
bool pedidos[4] = { false, false, false, false };
```

Assim, em vez de escrever diretamente em `andarDestino` cada vez que se carrega num botão, o código passou a:

- **registar o pedido nesse array,**
- deixando o cálculo de `andarDestino` para uma fase posterior, mais organizada.

3.2 Leitura dos botões usando o array

A parte do código responsável pelos botões passou de:

```
if (!digitalRead(pinBotaoA)) andarDestino = 1;
...
```

para:

```
if (!digitalRead(pinBotaoA)) pedidos[0] = true; // andar 1
if (!digitalRead(pinBotaoB)) pedidos[1] = true; // andar 2
if (!digitalRead(pinBotaoC)) pedidos[2] = true; // andar 3
if (!digitalRead(pinBotaoD)) pedidos[3] = true; // andar 4
```

Desta forma:

- Cada botão marca um índice do array `pedidos[]` como `true`.
- Cada índice representa um andar que **está em espera** para ser atendido.
- `andarDestino` deixa de ser modificado diretamente pelos botões, como o professor recomendou.

4. Cálculo do Próximo Andar de Destino

4.1 Função dedicada para calcular `andarDestino`

Para organizar o código e seguir a sugestão do professor (“pode ser uma função só para calcular isto, fica mais organizado”), foi criada uma função que decide qual o próximo andar de destino com base no array `pedidos[]` e no andar atual:

```
int calcularAndarDestino(int andarAtual) {  
  
    if (andarAtual == -1) return -1; // segurança  
  
    int proxAcima = -1;  
    int proxAbaixo = -1;  
  
    // procurar primeiro pedidos acima do andar actual  
    for (int i = andarAtual; i < 4; i++) {  
        if (pedidos[i]) {  
            proxAcima = i + 1;  
            break;  
        }  
    }  
  
    // se não houver acima, procurar pedidos abaixo  
    for (int i = andarAtual - 2; i >= 0; i--) {  
        if (pedidos[i]) {  
            proxAbaixo = i + 1;  
            break;  
        }  
    }  
  
    if (proxAcima != -1) return proxAcima;  
    if (proxAbaixo != -1) return proxAbaixo;  
  
    // se não houver pedidos, fica no mesmo andar  
    return andarAtual;  
}
```

4.2 Vantagens desta abordagem

- Se existirem pedidos **acima** do piso atual, o elevador continua a subir até os atender.
- Quando já não houver pedidos acima, pode então começar a descer para atender pedidos em andares inferiores.
- O comportamento é muito semelhante ao de um elevador real, que tende a minimizar movimentos desnecessários.

5. Gestão dos LEDs de Andar

5.1 Problema inicial

No comportamento original, os LEDs eram controlados em função de `andarDestino` e de algumas condições de paragem, o que causava:

- LEDs de vários andares a apagarem-se todos ao mesmo tempo quando o elevador chegava ao último andar pedido.
- Falta de correspondência clara entre LED aceso e “pedido pendente”.

5.2 Nova lógica baseada no array de pedidos

A lógica foi alterada para:

```
for (int i = 0; i < 4; i++) {  
    digitalWrite(ledAndar[i], pedidos[i] ? HIGH : LOW);  
}
```

Desta forma:

- Um LED aceso significa que **existe um pedido pendente** para esse andar.
- Quando o elevador passa nesse andar e o pedido é atendido (`pedidos[i] = false`), o LED apaga automaticamente no ciclo seguinte.
- Se alguém voltar a carregar no botão desse andar, o LED volta a acender (novo pedido registrado).

Isto resolve o conflito que explicaste:

“o elevador passou no terceiro, ele não apagou o pedido e depois chegou ao 4 e apagou os dois ao mesmo tempo”

Agora:

- Quando passa no 3, atende o pedido do 3 e apaga **só o LED do 3**.
- Depois segue para o 4, atende o pedido do 4 e apaga **só o LED do 4**.

6. Buzzer em Cada Paragem com Pedido

6.1 Problema inicial

Inicialmente, o buzzer só tocava quando o elevador parava no último destino, ou em condições específicas de travagem.

Não havia um toque consistente em **cada** andar pedido atendido.

6.2 Solução implementada

Quando o elevador entra num piso (detetado pela mudança de estado do sensor desse piso), o código verifica se havia um pedido para esse andar:

```
if (estadoSensorX && !estadoSensorXAnt) {  
    andar = X;  
    if (pedidos[X-1]) {  
        tone(pinBuzzer, 800);  
        delay(120);  
        noTone(pinBuzzer);  
    }  
    pedidos[X-1] = false;  
}
```

Assim:

- Se o andar tinha sido pedido, o buzzer toca.
- Em seguida, o pedido é apagado e o LED desse andar apaga.
- Se o andar não tinha pedido, o buzzer não toca.

Resultado prático:

- Se o elevador estiver no piso 1 e forem pedidos os pisos 3 e 4:
 - quando chega ao piso 3 → buzzer toca uma vez, LED do 3 apaga;
 - quando chega ao piso 4 → buzzer volta a tocar, LED do 4 apaga.
 - Isto corresponde ao comportamento desejado e ao que se vê nos elevadores reais.
-

7. Conclusão

A evolução do código permitiu:

1. **Resolver o erro principal** onde apenas o último pedido era guardado e os anteriores eram perdidos.
2. Implementar uma **variável intermédia sob a forma de array** (`pedidos[]`) para registar todos os andares pressionados/pendentes, tal como sugerido pelo professor.
3. Criar uma função específica para **calcular o andarDestino** com base nesses pedidos, tornando o código mais organizado e fácil de compreender.
4. Corrigir o comportamento dos **LEDs**, que agora representam fielmente os pedidos ativos.
5. Garantir que o **buzzer toca em cada piso pedido** à medida que o elevador vai atendendo as chamadas.

Estas alterações aproximam o sistema de controlo do elevador a um funcionamento real, melhoram a clareza do código e facilitam futuras expansões (como mais andares, chamadas interiores na cabine, controlo de portas, modos de prioridade, etc.).